

A Strategy for AI-Assisted Coding at Acme

TL;DR

AI coding tools have spread across Acme's engineering org for 18 months. Adoption is fast, value is low, and impact is invisible. The problem isn't which tool to pick. It's that there's no system around the tools at all.

Three things are broken at once:

- **No golden path.** Four-plus toolchains, no approved procurement route, and proprietary code likely flowing through consumer LLM accounts right now.
- **No enablement.** 76% use AI daily, but only 29% know how to use it well. Value is gated by fluency, not access.
- **No feedback loop.** Leadership can't see what works, for whom, or whether the spend returns anything.

The recommendation is a two-tier governed workspace: **GitHub Copilot Enterprise** as the platform layer (telemetry, security, full IDE breadth including mobile), with **Claude Code** as the named agentic tier for complex multi-step work, plus a governed **Hatchway** for frontier tools. The tools solve different problems; the governance layer is one. A portfolio differs from a pile of subscriptions by exactly one thing: a single control plane (one way to provision, shared telemetry, one golden path).

The lasting asset isn't the tool choice. It's the platform layer and the internal fluency corpus built on top of it (prompt libraries, MCP servers, scaffolds, a champion network). Those survive any vendor swap. So: invest in enablement in parallel, instrument from day one, and measure fluency rate and the productivity signal rather than licence count.

1. The Problem

Where AI coding sits

The developer journey runs Discover → Plan → Code → Validate → Deploy → Operate. AI coding tools live mainly at **Code**: completion, agentic generation, refactoring, inline Q&A. Their leverage spills both ways, though. A well-prompted agent speeds up ticket decomposition (Plan) and cuts review cycles (Validate) by producing better first drafts.

So the frame isn't "IDE productivity." It's **engineering velocity**: how fast working software moves from intent to production. AI is a multiplier on the most time-intensive stage of that chain.

Three breakdowns, all at once

Eighteen months of uncoordinated adoption produced a specific failure: the org bought adoption it can't convert and can't see that it failed. Three gaps cause it, and they reinforce each other.

1. No golden path. No standard, no central guidance, no approved way to buy. Tools spread by team preference and expense claims, leaving at least four toolchains. With only ~900 engineers on any trackable tool and 76% reporting daily AI use, a large group is almost certainly routing proprietary code through consumer-grade LLM interfaces. IP exposure isn't a future risk. It's happening now.

2. No enablement. Usage is high (76% daily), value capture is low (41% report meaningful gains, 29% feel they use their tool well). That's a fluency gap, not a tool-quality gap. Picking one vendor doesn't touch it. The same deficit persists under any tooling decision.

3. No feedback loop. Nothing measures adoption, usage quality, or impact with any confidence. Leadership can't tell which tools work or whether the investment returns anything, so the strategy can't self-correct and the spend can't be defended. This is the same gap as fragmentation, not a separate one: fragmentation is what makes measurement impossible. Consolidation's real prize isn't fewer tools. It's the measurement surface that makes every later decision improvable.

The three feed each other. Fragmentation blocks measurement. Weak measurement makes the enablement case hard to fund. Weak enablement keeps value low, which erodes confidence in the whole programme.

There's a fourth dynamic underneath. Speeding up the Code stage doesn't automatically raise velocity; it can push cost downstream. If AI code is faster to write but harder to review (only 33% feel equipped to validate AI output), Validate clogs and the net gain disappears. So success metrics have to move two baselines: the 41% productivity number **and** the 33% quality-confidence number, not just fluency and seat count.

2. Vendor Evaluation

Criteria and weights

Six criteria, weighted by the three breakdowns above and the mandate of the role.

Criterion	Weight	Why
Security / IP governance	20%	A strategy that leaks proprietary code isn't a strategy. Data residency, IP protection, enterprise controls are the floor; depth differentiates.
Enablement & ecosystem	20%	29% fluency is the binding constraint. Tool choice without enablement changes nothing.
Agentic capability	20%	The north star. Inline completion is table stakes; multi-step agent loops, MCP, CLI composability are the forward vector.
Visibility / telemetry	15%	The feedback loop is broken. No instrumentation, no investment case.
IDE breadth	10%	The real mix includes Xcode and Android Studio, not just VS Code. Mobile is currently excluded.
Adoption inertia	15%	Displacing tools engineers chose creates friction. Riding existing momentum is cheaper and faster.

Scorecard

Tool	Security	Enablement	Agentic	Visibility	IDE	Adoption	Weighted
GitHub Copilot Enterprise	5	4	3	5	5	3	4.10
Gemini Code Assist	5	4	2	3	3	5	3.70
Claude Code	4	3	5	2	2	4	3.50
Cursor (Enterprise)	3	2	5	2	3	2	2.90

Recommendation: two tiers plus a Hatchway

Tier 1, platform layer: GitHub Copilot Enterprise. It wins on the three criteria tied to our breakdowns: best telemetry (Copilot Metrics API fixes the feedback loop), widest IDE coverage including Xcode (fixes mobile exclusion), and enterprise security at scale. Its multi-model support matters most: Claude and Gemini models

are selectable in-IDE, so the ~400 Gemini-happy backend engineers keep their preferred model. That's the migration unlock (validate parity on the highest-value flows before migrating). Gemini's higher raw adoption (400 vs 120) is real, but it's preserved here, not lost. The 4.10-vs-3.70 gap comes from visibility, IDE breadth, and agentic trajectory.

Tier 2, agentic layer: Claude Code (Anthropic Enterprise). Highest agentic score (5/5), and already the tool platform and frontend engineers reached for on hard tasks. A model picker isn't an agent harness. Claude Code is a CLI agent loop with MCP-native composability: filesystem access, multi-step orchestration, tool use across the repo. Copilot's picker exposes the same weights; it doesn't expose that runtime. The move is to centralise the existing ~300 users under an Anthropic Enterprise contract (central billing, API-level usage tracking, admin controls), not to change the tool. Organic adoption already passed the product-market-fit test for free. These users also become the main internal enablement resource for everyone else, and the tier directly serves the role's mandate to build next-gen agentic workspaces.

Tier 3, the Hatchway: a governed exception process. A named route for anything outside Tiers 1 and 2. Current candidates: Cursor (~60), untracked self-hosted or open-source tools, and future frontier tools. Entry requires a passed security review, wired-up telemetry, a budget-capped 90-day trial, and a named engineering lead. Anything operating outside the Hatchway is shadow IT.

3. Strategy

Enablement at scale

29% fluency is the constraint, and it survives any tool decision. Fix the golden path without fixing fluency and you get a governed version of the same low-value state.

Infrastructure. A self-service IDP page is the default entry point for anyone onboarding to AI tooling: new hires, teams adopting a first tool, and every Hatchway request. Minimum version: a curated guide pointing to approved tools, prompt libraries, and MCP configs. Target version: self-service provisioning by template (tool access, pre-set IDE config, prompts for the team's stack). Where there's no IDP, a curated repo is the fallback. Either way, a skill/MCP layer exposes the same resources inside the IDE or CLI, so there's no context switch to a browser. The forcing function is provisioning itself: when an engineer gets access, they get pre-configured settings, a prompt pack for their stack, and the right MCP servers on day one. You raise fluency by shipping it, not by scheduling training.

Social. Sharing sessions, best-practice guides, and internal case studies are supporting signal, not the main mechanism. Claude Code power users, recruited as advocates, pull adoption through peer demonstration instead of a top-down mandate.

Platform engineering owns this. The PM sets direction, defines the resource taxonomy, and owns the metrics; platform engineers build and run the infrastructure.

Reaching the ~3,700 non-users

The population isn't homogeneous. It splits into three groups: never had access or never tried; tried and bounced (friction, poor fit, no help); and using ungoverned AI right now through personal accounts. The third group is urgent because it's already leaking IP. For them the goal is migration from ungoverned to governed, not recruitment from zero.

Detection comes before rollout: expense audits (personal AI subscriptions on company cards), egress monitoring for known LLM endpoints, and a targeted follow-up survey to the 76%-daily cohort. Without a detection signal the "3,700" is decoration; you can't run a funnel you can't see into.

Rollout sequence: power users and early adopters first (confirm the governed platform doesn't add friction where value already exists), then low-satisfaction current users (the highest-value conversions), then full rollout with comms.

Mobile engineers. Previously locked out by tooling gaps. Copilot's Xcode and Android Studio support opens a path, but mobile is its own workstream, not a vendor-selection fix. Context quality and custom MCP for Swift/Kotlin need deliberate investment. For an ecommerce and delivery company whose app is the primary revenue surface, treating mobile AI tooling as an afterthought is a business risk. So the IDP ships mobile resources from day one (Swift and Kotlin prompt packs, mobile testing patterns, the right MCP configs), and the behaviour-change hook is a peer showcase: a mobile engineer demoing a real Xcode workflow. Mobile joins the same rollout cohort but is tracked as its own segment to confirm the resources land.

The existing fragmented base

Before the platform is ready. Send policy comms to all current AI users: a clear, short note on what not to paste into any AI tool while governance is stood up. Cheap, immediate, and aimed straight at the known IP window.

Gemini (~400). Frame migration to Copilot around the org win: standard tooling, consolidated telemetry, network effects from a shared platform. The reassurance is practical: Copilot is multi-model, so they keep their model; validate the top Gemini Code Assist flows for parity first. Only the governance around the model changes.

Claude Code (~300). Organic adopters on the highest-capability tier. Not a problem to suppress, the primary enablement resource. Centralise under Anthropic Enterprise (billing, admin, API-level tracking) and run showcase sessions where they demo their setups. Advocates, not detractors.

Cursor (~60). Not banned, removed from the golden path. Access goes through the Hatchway: state the case, pass security review, 90-day capped trial, named lead. Same model as any tool outside the two tiers.

Measuring success

Prerequisite (days 0-60). Wire up the Copilot Metrics API and set a PR cycle-time baseline before the first wave. No baseline, no measurable 12-month signal. This is a sequencing dependency, not a nice-to-have.

Horizon	Target
3 months	No usage regression among active users. ≥50% of active users migrated to the governed platform. Telemetry live and reporting.
6 months	Fluency measured via pulse survey against the 29% baseline; target a directional lift. Usage analytics (feature depth, range of capabilities used) as the behavioural check on self-report. Neither alone suffices.
12 months	Higher feature velocity without quality regression: DORA lead time for changes and change failure rate against baseline, with PR cycle time as supporting signal. These are proxies and need paired qualitative input (EM surveys) to read correctly. PR volume alone is invalid; AI inflates it trivially.

Engineering leadership owns the programme. The PM drives execution: roadmap, resource taxonomy, metric definitions, stakeholder reporting.

4. Risks & Trade-offs

Consolidation without enablement still yields low value. The vendor pick fixes governance and measurement. It doesn't move 29% fluency. Delay or deprioritise enablement and you get a better-

instrumented version of the same low-value state.

Shadow IT during transition. Between the policy comms and full platform readiness, people keep using ungoverned tools. The interim comms reduces this; it doesn't end it. Treat it as a known, time-boxed exposure.

Mobile exclusion at launch. If the IDP ships without Swift/Kotlin packs and mobile-tested MCP, mobile engineers adopt generic patterns or nothing. The mobile pack is a launch dependency.

EM resistance to instrumentation. Telemetry can read as surveillance. Instrument tools, not individuals; report at the aggregate team level. Set that framing early or lose EM cooperation.

GCP procurement relationship. Deprioritising Gemini Code Assist may be commercially sensitive if Delivery Hero has a strategic GCP relationship. The recommendation doesn't drop Gemini; it consolidates under Copilot while keeping Gemini model access in-IDE. Make that explicit in any vendor conversation to avoid a procurement escalation.

Budget uncertainty. Two tiers mean concurrent contract costs. Mitigation is phased contracting: Copilot Enterprise first (broadest coverage, highest telemetry value), Anthropic Enterprise once the pilot proves out. Provision seats against adoption cohorts, not in bulk, so spend tracks value. That turns budget uncertainty into a designed lever: start with the known active base (~700), prove the signal, expand. Exact sequencing depends on budget clarity the brief doesn't give.

What we're explicitly not doing

Not doing	Why
Standardising on Gemini despite top adoption	Wins adoption inertia; loses on visibility, Xcode/mobile, and agentic trajectory.
Full single-tool mandate from day one	Forces underground adoption; burns goodwill with engineers who chose tools for real reasons.
Keeping Cursor as a named tier	Security risk, weakest telemetry, excludes non-VS Code engineers at scale. The Hatchway covers the legitimate need.
Big-bang rollout to all ~5,000	Unmanageable support load; no signal on what works before it's too late to correct.
Measuring productivity before tooling is stable	Measuring mid-transition produces noise and a false baseline.
Building custom self-hosted tooling	Engineering cost and velocity risk with no clear edge over the recommended tiers.